Product Requirements Document

ROS — Rubin Operating System

File-System-Based Multi-Agent Operating System for Claude Co-Work

Version: 1.0
Owner: Guy Rubin
Status: Draft for architecture review
Primary Platforms: Claude Co-Work, Obsidian, Notion, Google Workspace
Core Principle: One role = one plugin. One plugin = one responsibility. One command = one repeatable workflow.

---

1. Executive Summary

ROS — Rubin Operating System — is a file-system-based, tool-aware, multi-LLM AI operating system designed to coordinate personal execution, enterprise architecture consulting, real estate development, AI product creation, marketing, and finance/admin work through a modular multi-agent model.

ROS is not a single assistant and not one large plugin. It is a role-based operating system where each major role is implemented as a separate plugin with its own instructions, memory, skills, commands, templates, workbooks, connectors, dashboards, assets, and audit trail.

The system uses:

Layer	Platform	Purpose
Reasoning Layer	Claude Co-Work + optional LLMs	Orchestration, analysis, planning, generation
Knowledge Layer	Obsidian	Markdown-based memory, principles, templates, research, role instructions
Operating Layer	Notion	Dashboards, registries, tasks, decisions, assets, operational state
Execution Layer	Google Workspace	Gmail, Calendar, Drive, Docs, Sheets, Slides
Tool Layer	Connectors/APIs/MCP/tools	Access to systems, diagnostics, automation, data retrieval
Code & Versioning Layer	GitHub	Source control, issues, branches, PRs, releases, CI/CD, environment configuration

Layer	Platform	Purpose
Environment Layer	Dev/stage/prod/sandbox runtimes	Safe execution spaces for agents, tools, automations, and infrastructure
LLM Runtime Layer	Claude Co-Work, Claude Code, Hermes, GPT, local models, specialist models	Pluggable model execution by role, command, or skill
Sync Layer	GitHub + Obsidian + Notion + Google Workspace sync jobs	Keeps file-system agents, dashboards, assets, and execution state aligned without turning the system into chaos

ROS must support multiple identities, multiple business domains, multiple clients, reusable templates, command-based workflows, workbook-driven decisions, and strict confirmation gates for external actions.

---

2. Problem Statement

Current AI co-worker designs often fail because they collapse too much responsibility into one generic agent or one generic plugin. This creates five problems:

1. **Role confusion** — the system does not know whether it is acting as Chief of Staff, Personal Assistant, Real Estate Analyst, Enterprise Architect, Product Manager, Marketer, or Finance/Admin operator.
2. **Identity risk** — Gmail, Google Workspace, and business accounts can be mixed incorrectly.
3. **Context leakage** — enterprise architecture work across multiple companies can blend client-specific information.
4. **Operational chaos** — tasks, dashboards, assets, and decisions are not consistently captured.
5. **Bloated instructions** — Markdown files become long, repetitive, and hard for the AI to use effectively.

ROS solves this by using a clean, modular architecture: roles become plugins, repeatable workflows become commands, reusable capabilities become skills, numeric decisions become workbooks, and operational state is managed through Notion.

---

3. Product Vision

Create a professional-grade AI operating system that behaves like a small executive team:

- **COS** acts as Chief of Staff and owns strategy, prioritization, agent design, and operating cadence.
- **KK** acts as Personal Assistant and owns daily execution, inbox, calendar, follow-ups, and cross-domain coordination.
- **HV** owns HollandVest real estate development.
- **EA** owns enterprise architecture consulting across multiple companies and roles.
- **PAI** owns AI product strategy and product build workflows.

- **MKT** owns marketing, content, campaigns, and distribution.
- **FIN** owns finance, admin, insurance, contracts, invoices, and compliance packs.

The result should be a system that is:

- Modular
- Fast to use
- Safe by design
- Dashboard-driven
- Template-first
- Command-based
- Workbook-enabled
- Identity-aware
- Client-confidentiality aware
- Built for execution, not theory

---

4. Product Goals

4.1 Primary Goals

1. Build ROS as a file-system-based multi-agent operating system for Claude Co-Work.
2. Implement each major role as a separate plugin.
3. Use Obsidian as the durable knowledge and instruction layer.
4. Use Notion as the operational dashboard and database layer.
5. Use Google Workspace as the execution and collaboration layer.
6. Support multiple Gmail and Google Workspace identities with strict routing.
7. Support multi-client enterprise architecture work with strict client separation.
8. Convert repeatable workflows into slash-style commands.
9. Convert numeric/scoring decisions into workbooks.
10. Maintain clear registries for plugins, skills, commands, assets, connectors, and workbooks.
11. Enforce smart and efficient Markdown instruction principles.
12. Require confirmation before external actions such as sending emails, submitting forms, sharing documents, or deleting files.
13. Include a tool diagnostic layer that validates connector health, identity routing, permissions, command readiness, and tool failures.
14. Support hybrid LLM execution, where different models can work on the same file-system-based agent framework without breaking role boundaries or memory governance.
15. Integrate GitHub as the source-control and collaboration backbone for ROS file-system agents, commands, skills, templates, workbooks, and infrastructure definitions.
16. Add environment management for local, sandbox, development, staging, and production execution so agents and tools can be tested safely before being used operationally.
17. Treat Hermes as a first-class contributor and runtime for the agent system, especially for KK/Personal Assistant workflows and operational automation.
18. Add a sync architecture so GitHub, Obsidian, Notion, Google Workspace, Claude Code, Claude Co-Work, and Hermes can work against the same controlled source of truth.

19. Support scheduled synchronization jobs, including GitHub backup, Notion dashboard updates, Obsidian publishing queues, and Google Drive asset publishing.

4.2 Non-Goals

ROS must not:

- Be implemented as one plugin.
- Use one generic assistant for all domains.
- Mix personal, HollandVest, and enterprise architecture identities.
- Mix client data across enterprise architecture engagements.
- Treat Notion as unstructured notes.
- Treat Obsidian as a task manager.
- Treat Google Drive as the system of record for operational state.
- Generate long, bloated Markdown instruction files.
- Send, submit, delete, publish, or share externally without explicit confirmation.

---

5. Design Principles

5.1 Role Separation

Each major role must be implemented as a separate plugin.

```
One role = one plugin.
One plugin = one folder.
One folder = one responsibility.
```

5.2 Platform Separation

```
Obsidian = thinking, knowledge, principles, reusable context.
Notion = operations, dashboards, tasks, registries, decisions.
Google Workspace = execution, communication, collaboration, files.
Claude Co-Work = reasoning, orchestration, generation.
```

5.3 Command-First Execution

Every repeatable workflow should become a command.

Examples:

```
/cos.weekly-review
/kk.daily-plan
/hv.analyze-deal
```

```
/ea.write-hld
/pai.create-prd
/mkt.create-campaign
/fin.prepare-compliance-pack
```

5.4 Template-First Output

If a repeatable output exists, never start from blank.

Templates must exist for:

- Weekly reviews
- Meeting briefs
- Deal memos
- Investment scorecards
- HLDs
- ADRs
- PRDs
- Campaign plans
- Contract summaries
- Compliance packs

5.5 Workbook-Driven Decisions

Use a workbook when the decision involves:

- Money
- Risk
- Scoring
- Ranking
- Scenarios
- Forecasting
- Cost modeling
- Prioritization

5.6 Smart & Efficient Markdown

All Markdown and instruction files must be concise, modular, and execution-oriented.

Core rule:

```
Every instruction must earn its place.
If it does not improve execution, remove it.
```

5.7 Identity Safety

Before any Google Workspace action, ROS must identify:

1. Acting plugin
2. Sender or workspace identity
3. Target recipient/system
4. Purpose
5. Safety level
6. Confirmation requirement

5.8 Client Confidentiality

EA must always identify client context before using client-specific memory, stakeholders, documents, risks, decisions, or architecture material.

If client context is unclear, EA must default to generic-career or request clarification.

5.9 Tool-Aware by Design

ROS must know which tools, connectors, APIs, identities, and permissions are available before attempting execution.

Tool use must follow this rule:

```
Diagnose first.
Act second.
Escalate if blocked.
```

Before using a connector, ROS should know:

- Tool name
- Plugin allowed to use it
- Identity/account required
- Permission status
- Last known health status
- Supported actions
- Safety level
- Fallback option

5.10 Hybrid LLM by Design

ROS must support more than one LLM working on the same file-system-based agent architecture.

Claude Co-Work can be the primary orchestrator, Claude Code can be the implementation/runtime engineer, and Hermes can be a first-class operational contributor. Other models may be assigned to specific roles, skills, or commands.

Examples:

```
Claude = reasoning, orchestration, writing, architecture
Hermes = personal assistant execution, local automation, lightweight daily ops
GPT = research, drafting, synthesis, coding support
Local LLM = private/offline summarization or classification
Specialist model = code, OCR, embeddings, retrieval, financial extraction
```

The file system, not the model, is the durable source of truth.

5.11 GitHub-Backed Operating System

ROS must be version-controlled in GitHub.

GitHub is the engineering control plane for:

- Source control for all plugin files
- Branching for new agents, commands, and skills
- Pull requests for reviewable changes
- Issues for backlog and defects
- Releases for stable ROS versions
- GitHub Actions for validation and checks
- Environment files for local/dev/stage/prod configuration
- Auditability of changes to agent behavior

5.12 Environment-Managed Execution

ROS must separate environments so agents can be tested before they act on real systems.

Required environments:

```
local = private development and file editing
sandbox = safe tool simulation and dry runs
dev = connected test integrations
stage = pre-production validation
prod = real operational use
```

No new command, connector, or automation should be promoted to production without passing diagnostic checks in a lower environment.

5.13 Sync by Design

ROS must keep GitHub, Obsidian, Notion, Google Workspace, Claude Co-Work, Claude Code, and Hermes aligned through controlled sync lanes.

Core sync rule:

```
GitHub versions the system.
Obsidian stores thinking and durable knowledge.
Notion shows operational state.
Google Workspace stores final collaboration assets.
Hermes runs personal assistant operations and scheduled jobs.
Claude Code implements repository changes.
Claude Co-Work orchestrates strategy and reasoning.
```

Sync must be explicit, logged, and environment-aware. No silent two-way sync for MVP.

5.14 Contributor Model

ROS must support multiple contributors working safely on the same file-system agent architecture.

Contributor types:

```
Guy = owner and final approver
Claude Co-Work = strategy/orchestration contributor
Claude Code = implementation/repository contributor
Hermes = operational/runtime contributor
GPT/local models = specialist contributors
GitHub = collaboration and review layer
```

All non-trivial changes to plugins, commands, skills, templates, runtime rules, tool access, or environment behavior must go through GitHub branches and pull requests unless explicitly authorized.

---

6. Key Definitions

Term	Definition	Example
Role	AI worker identity with responsibility	COS, KK, HV, EA
Plugin	Executable file-system package for one role	`/plugins/HV/`
Skill	Reusable capability inside a plugin	`deal-analyzer`, `adr-writer`
Command	Named repeatable workflow	`/hv.analyze-deal`

Term	Definition	Example
Workbook	Structured decision model	BRRRR model, DSCR model
Connector	External system access	Gmail, Notion, Drive
Asset	Durable output	PRD, HLD, investment memo
Dashboard	Operational control panel	HV Deal Pipeline
Registry	Notion database tracking system components	Plugin Registry
Memory	Durable context for role, client, or domain	`MEMORY.md`

---

7. System Architecture

7.1 High-Level Architecture

```
/ROS
│
├── 00_SYSTEM
│   ├── principles.md
│   ├── routing.md
│   ├── role-hierarchy.md
│   ├── plugin-policy.md
│   ├── skill-policy.md
│   ├── command-policy.md
│   ├── workbook-policy.md
│   ├── connector-policy.md
│   ├── identity-policy.md
│   ├── memory-policy.md
│   ├── notion-policy.md
│   ├── obsidian-policy.md
│   ├── google-workspace-policy.md
│   ├── markdown-instruction-principles.md
│   └── audit-policy.md
│
├── 01_ROOT_PLUGINS
│   ├── COS
│   └── KK
│
├── 02_DOMAIN_PLUGINS
│   ├── HV
│   ├── EA
│   ├── PAI
│   ├── MKT
```

```
│   └── FIN
│
├── 03_SHARED
│   ├── skills
│   ├── commands
│   ├── templates
│   ├── workbooks
│   ├── connectors
│   └── prompts
│
├── 04_ASSETS
│   ├── artifacts
│   ├── workbooks
│   ├── dashboards
│   ├── reports
│   ├── emails
│   └── documents
│
├── 05_AUDIT
│   ├── session-logs
│   ├── command-runs
│   ├── decision-logs
│   └── change-logs
│
├── 06_TOOL_DIAGNOSTICS
│   ├── tool-registry.md
│   ├── connector-health.md
│   ├── identity-checks.md
│   ├── permission-checks.md
│   ├── failure-log.md
│   └── fallback-routes.md
│
├── 07_LLM_RUNTIME
│   ├── model-registry.md
│   ├── routing-rules.md
│   ├── model-capabilities.md
│   ├── model-permissions.md
│   ├── handoff-protocol.md
│   └── evaluation-log.md
│
├── 08_GITHUB
│   ├── repository-policy.md
│   ├── branching-strategy.md
│   ├── pull-request-policy.md
│   ├── issue-labels.md
│   ├── release-policy.md
```

```
│   ├── github-actions.md
│   └── codeowners.md
│
├── 09_ENVIRONMENTS
│   ├── environment-registry.md
│   ├── local.env.example
│   ├── sandbox.env.example
│   ├── dev.env.example
│   ├── stage.env.example
│   ├── prod.env.example
│   ├── secrets-policy.md
│   ├── promotion-policy.md
│   └── environment-diagnostics.md
│
└── 10_SYNC
    ├── sync-policy.md
    ├── source-of-truth-map.md
    ├── github-sync.md
    ├── obsidian-sync.md
    ├── notion-sync.md
    ├── google-workspace-sync.md
    ├── hermes-cron-jobs.md
    ├── conflict-resolution.md
    └── sync-audit-log.md
```

7.2 Role Hierarchy

```
Guy
│
└── ROS Root
    │
    ├── Root Plugins
    │   ├── COS — Chief of Staff
    │   └── KK — Personal Assistant
    │
    ├── Domain Plugins
    │   ├── HV — HollandVest Real Estate Development
    │   ├── EA — Enterprise Architecture Consulting
    │   ├── PAI — AI Product Team
    │   ├── MKT — Marketing / Content
    │   └── FIN — Finance & Admin
    │
    ├── Shared Layer
    │   ├── Shared Skills
    │   ├── Shared Commands
```

```
    │   ├── Shared Templates
    │   ├── Shared Workbooks
    │   └── Shared Connectors
    │
    └── Control Plane
        ├── Notion Registries
        ├── Dashboards
        ├── Audit Logs
        └── Asset Library
```

---

8. Plugin Requirements

Every plugin must include:

```
CLAUDE.md
MEMORY.md
skills/
commands/
templates/
workbooks/
connectors/
dashboards/
assets/
audit/
```

8.1 Plugin File Standards

File/Folder	Purpose
`CLAUDE.md`	Role, mission, boundaries, commands, output rules
`MEMORY.md`	Durable facts, context, preferences, decisions
`skills/`	Reusable capabilities
`commands/`	Repeatable workflows
`templates/`	Standard output formats
`workbooks/`	Decision models
`connectors/`	Allowed external systems and identity mappings
`dashboards/`	Notion dashboard definitions
`assets/`	Durable outputs owned by the plugin

File/Folder	Purpose
`audit/`	Command runs, decisions, and change history

8.2 Plugin Operating Rule

Plugins may use shared skills and shared templates, but they must preserve their own domain context and cannot perform work outside their responsibility unless delegated by COS or KK.

---

9. Root Plugins

9.1 COS — Chief of Staff Plugin

Purpose

COS owns strategy, prioritization, operating rhythm, agent design, and system governance.

Responsibilities

- Manage the overall ROS operating model.
- Create and improve role plugins.
- Define new skills, commands, and templates.
- Prioritize work across domains.
- Run weekly reviews.
- Run monthly portfolio reviews.
- Identify blockers and risks.
- Decide what should be automated next.
- Maintain agent, skill, command, and asset registries.
- Challenge low-value work.
- Translate strategic goals into execution roadmaps.

COS Folder Structure

```
/01_ROOT_PLUGINS/COS/
  CLAUDE.md
  MEMORY.md

  skills/
    strategy-review.md
    priority-ranking.md
    agent-designer.md
    operating-rhythm.md
    risk-review.md
    decision-review.md
```

```
  commands/
    weekly-review.md
    monthly-review.md
    design-new-agent.md
    review-agent-performance.md
    prioritize-week.md
    decide-next-automation.md

  templates/
    strategy-memo.md
    weekly-review.md
    monthly-portfolio-review.md
    agent-spec.md
    decision-memo.md

  dashboards/
    command-center.md
    agent-health.md
    priority-board.md

  assets/
    strategy-memos/
    operating-models/
    agent-specs/
    decision-logs/
```

COS Commands

```
/cos.weekly-review
/cos.monthly-portfolio-review
/cos.prioritize-week
/cos.design-agent
/cos.review-agent
/cos.create-skill
/cos.kill-or-defer
/cos.decide-next-automation
```

COS Acceptance Criteria

COS is successful when it can:

- Produce a weekly priority plan.
- Review plugin performance.
- Recommend new commands or skills.
- Identify blockers across domains.
- Update the operating model without bloating files.

- Maintain system discipline.

9.2 KK — Personal Assistant Plugin

Purpose

KK owns daily execution, operational planning, inbox, calendar, follow-ups, document collection, and cross-domain coordination.

Responsibilities

- Daily planning.
- Calendar review.
- Inbox triage.
- Follow-up tracking.
- Meeting preparation.
- Document collection.
- Task coordination.
- Cross-domain reminders.
- Voice note extraction.
- Prepare draft emails.
- Update operational dashboards.
- Ask domain plugins for specialized input.

Boundary Rule

```
KK coordinates.
COS prioritizes.
HV analyzes real estate.
EA handles architecture.
PAI designs products.
MKT handles content.
FIN handles admin and compliance.
```

KK Folder Structure

```
/01_ROOT_PLUGINS/KK/
  CLAUDE.md
  MEMORY.md

  skills/
    inbox-manager.md
    calendar-manager.md
    follow-up-tracker.md
```

```
    daily-planner.md
    meeting-prep.md
    voice-extract.md
    task-extractor.md
    email-drafter.md

  commands/
    daily-plan.md
    inbox-triage.md
    prepare-meeting.md
    follow-up-review.md
    collect-documents.md
    create-task-list.md
    coordinate-domain-agent.md

  templates/
    daily-plan.md
    follow-up-list.md
    meeting-brief.md
    document-checklist.md
    email-draft.md

  dashboards/
    personal-command-center.md
    waiting-for.md
    today.md

  assets/
    daily-plans/
    meeting-briefs/
    follow-up-lists/
    document-checklists/
```

KK Commands

```
/kk.daily-plan
/kk.inbox-triage
/kk.follow-up-review
/kk.prepare-meeting
/kk.collect-documents
/kk.coordinate-with-hv
/kk.coordinate-with-ea
/kk.coordinate-with-pai
/kk.coordinate-with-fin
```

KK Acceptance Criteria

KK is successful when it can:

- Produce a daily plan from calendar, inbox, tasks, and priorities.
- Track waiting-for items.
- Prepare meeting briefs.
- Draft emails using the correct identity.
- Coordinate with domain plugins without replacing them.
- Surface missing documents and open follow-ups.

---

10. Domain Plugins

10.1 HV — HollandVest Real Estate Development Plugin

Purpose

HV owns HollandVest real estate development, BRRRR analysis, asset ranking, financing, permits, WWS risk, vendor coordination, and investment decision support.

Responsibilities

- Analyze Dutch real estate assets.
- Evaluate BRRRR potential.
- Assess unit split and volume development upside.
- Review permit pathway.
- Analyze WWS and rent regulation risk.
- Build lender packs.
- Draft messages to brokers, lenders, architects, and contractors.
- Maintain HollandVest Notion dashboard.
- Build workbooks for deal decisions.

HV Folder Structure

```
/02_DOMAIN_PLUGINS/HV/
  CLAUDE.md
  MEMORY.md

  skills/
    deal-analyzer.md
    brrrr-calculator.md
    permit-pathway.md
    wws-analyzer.md
    financing-pack-builder.md
    vendor-comparison.md
```

```
  architect-brief-writer.md
  lender-email-writer.md

commands/
  analyze-deal.md
  rank-assets.md
  build-brrrr-workbook.md
  build-dscr-workbook.md
  assess-permit-pathway.md
  prepare-lender-pack.md
  prepare-architect-brief.md
  update-deal-dashboard.md

workbooks/
  brrrr-model.md
  dscr-ltv-model.md
  renovation-budget.md
  vendor-comparison.md
  wws-impact-model.md
  permit-scenario-model.md

templates/
  deal-scorecard.md
  investment-memo.md
  lender-request.md
  architect-brief.md
  contractor-rfq.md

connectors/
  notion-hollandvest.md
  gmail-bhollandvest.md
  google-drive-hollandvest.md
  funda.md
  funda-business.md
  kadaster.md
  omgevingsloket.md

dashboards/
  deal-pipeline.md
  financing-dashboard.md
  permit-dashboard.md
  renovation-dashboard.md

assets/
  deal-memos/
  workbooks/
  lender-packs/
```

```
    architect-briefs/
    vendor-comparisons/
```

HV Commands

```
/hv.analyze-deal
/hv.rank-assets
/hv.build-brrrr-workbook
/hv.build-dscr-model
/hv.assess-permit-risk
/hv.prepare-lender-pack
/hv.prepare-architect-email
/hv.update-notion-deal
```

HV Workbooks

- BRRRR Deal Model
- DSCR / LTV Model
- Renovation Budget Tracker
- Vendor Comparison Workbook
- WWS Impact Model
- Permit Scenario Model

HV Acceptance Criteria

HV is successful when it can:

- Produce a deal memo from a listing.
- Rank multiple assets by development upside and risk.
- Build a BRRRR workbook.
- Identify financing gaps.
- Draft lender and architect emails.
- Update the Notion deal pipeline.

10.2 EA — Enterprise Architecture Consulting Plugin

Purpose

EA owns enterprise architecture consulting across multiple companies, multiple roles, and multiple client contexts.

EA must be multi-client and confidentiality-safe.

Responsibilities

- Support architecture work across multiple clients.
- Write HLDs, LLDs, ADRs, and architecture reviews.
- Prepare client meetings.
- Build cloud governance models.
- Review security and compliance risks.
- Prepare interview and role material.
- Maintain client-specific architecture memory.
- Prevent client data leakage.

EA Client Model

```
/02_DOMAIN_PLUGINS/EA/clients/
  bpost/
    MEMORY.md
    stakeholders.md
    hlds/
    adrs/
    risks/
    meetings/
    decisions/

  kbc/
    MEMORY.md
    stakeholders.md
    hlds/
    adrs/
    risks/
    meetings/
    decisions/

  abn-amro/
    MEMORY.md
    interview-prep/
    scenarios/
    notes/

  european-commission/
    MEMORY.md
    stakeholders.md
    hlds/
    adrs/
    risks/
    meetings/
    decisions/
```

```
  generic-career/
    cv/
    linkedin/
    job-pipeline/
    interview-bank/
    architecture-patterns/
```

EA Folder Structure

```
/02_DOMAIN_PLUGINS/EA/
  CLAUDE.md
  MEMORY.md

  clients/
    bpost/
    kbc/
    abn-amro/
    european-commission/
    generic-career/

  skills/
    hld-writer.md
    adr-writer.md
    architecture-review.md
    cloud-governance-review.md
    security-risk-review.md
    platform-engineering-advisor.md
    interview-prep.md
    cv-optimizer.md
    client-brief-builder.md

  commands/
    write-hld.md
    write-adr.md
    review-architecture.md
    prepare-client-meeting.md
    create-risk-register.md
    prepare-interview.md
    update-client-dashboard.md
    create-cloud-governance-model.md

  workbooks/
    cloud-cost-model.md
    migration-wave-plan.md
    app-portfolio-assessment.md
    security-control-mapping.md
```

```
    vendor-comparison.md

  templates/
    hld.md
    adr.md
    risk-register.md
    security-review.md
    client-meeting-brief.md
    interview-brief.md

  connectors/
    notion-ea.md
    google-drive-ea.md
    gmail-bguy.md
    web-research.md
    github.md

  dashboards/
    client-dashboard.md
    architecture-decision-board.md
    risk-dashboard.md
    job-pipeline.md

  assets/
    hlds/
    adrs/
    reviews/
    client-briefs/
    interview-prep/
    cv/
```

EA Commands

```
/ea.write-hld --client=bpost
/ea.write-adr --client=kbc
/ea.prepare-client-meeting --client=bpost
/ea.review-architecture --client=generic
/ea.prepare-interview --client=abn-amro
/ea.create-risk-register --client=kbc
/ea.update-client-dashboard --client=bpost
```

EA Hard Rule

```
EA must always identify client context before using client-specific information.
Never mix client memory, stakeholders, risks, architecture decisions, documents,
```

```
or strategy between clients.
If the client is unclear, route to generic-career or ask for clarification.
```

EA Acceptance Criteria

EA is successful when it can:

- Produce client-specific HLDs and ADRs.
- Maintain separated client knowledge.
- Prepare client meeting briefs.
- Build architecture risk registers.
- Support interview preparation without leaking client-specific context.
- Update EA dashboards by client.

10.3 PAI — AI Product Team Plugin

Purpose

PAI owns AI product strategy, PRDs, agent architecture, MVP planning, technical backlog, monetization, and build specifications.

Responsibilities

- Create PRDs.
- Design AI agents and workflows.
- Define MVP scope.
- Build feature backlog.
- Select tech stack.
- Define monetization model.
- Create go-to-market plan.
- Prepare build instructions for Claude Code, Cursor, or engineering teams.
- Maintain product dashboards.

PAI Folder Structure

```
/02_DOMAIN_PLUGINS/PAI/
  CLAUDE.md
  MEMORY.md

  skills/
    prd-generator.md
    agent-architect.md
    mvp-planner.md
    roadmap-builder.md
    feature-prioritizer.md
```

```
  pricing-strategy.md
  gtm-strategist.md
  build-spec-writer.md

commands/
  create-prd.md
  design-agent-system.md
  define-mvp.md
  build-roadmap.md
  create-feature-backlog.md
  create-pricing-model.md
  create-build-spec.md

workbooks/
  feature-prioritization.md
  pricing-model.md
  market-sizing.md
  build-vs-buy.md

templates/
  prd.md
  agent-spec.md
  mvp-plan.md
  roadmap.md
  build-spec.md
  investor-memo.md

connectors/
  notion-ai-products.md
  github.md
  google-drive-products.md
  web-research.md

dashboards/
  product-roadmap.md
  build-backlog.md
  experiment-board.md

assets/
  prds/
  agent-specs/
  roadmaps/
  build-specs/
  pricing-models/
```

PAI Commands

```
/pai.create-prd
/pai.design-agent
/pai.define-mvp
/pai.create-roadmap
/pai.build-feature-matrix
/pai.create-pricing-model
/pai.write-build-spec
```

PAI Acceptance Criteria

PAI is successful when it can:

- Produce a professional PRD.
- Convert an idea into an MVP plan.
- Generate feature prioritization.
- Create build specs for implementation.
- Maintain a product roadmap in Notion.

10.4 MKT — Marketing / Content Plugin

Purpose

MKT owns content, campaigns, brand voice, sales copy, landing pages, newsletters, and distribution.

Responsibilities

- Create LinkedIn content.
- Write landing pages.
- Build newsletters.
- Plan campaigns.
- Repurpose content.
- Maintain content calendar.
- Create course content.
- Track campaign performance.

MKT Commands

```
/mkt.create-linkedin-post
/mkt.create-campaign
/mkt.write-landing-page
/mkt.create-newsletter
/mkt.build-content-calendar
```

```
/mkt.repurpose-content
/mkt.review-brand-voice
```

MKT Acceptance Criteria

MKT is successful when it can:

- Generate content aligned to the relevant brand.
- Maintain a content calendar.
- Build launch campaigns.
- Repurpose long-form assets into LinkedIn, newsletter, and landing-page formats.

10.5 FIN — Finance & Admin Plugin

Purpose

FIN owns invoices, insurance, contracts, company documents, bank onboarding, tax/admin evidence, and compliance packs.

Responsibilities

- Track invoices.
- Track insurance policies.
- Summarize contracts.
- Maintain company documents.
- Prepare bank onboarding packs.
- Collect tax/admin evidence.
- Build compliance packs.
- Draft admin emails.

FIN Commands

```
/fin.track-insurance
/fin.prepare-bank-documents
/fin.summarize-contract
/fin.create-invoice-tracker
/fin.build-compliance-pack
/fin.prepare-admin-email
```

FIN Acceptance Criteria

FIN is successful when it can:

- Track document requirements.

- Maintain insurance and contract status.
- Build compliance packs.
- Draft admin emails using the correct identity.
- Surface missing documents and expiry risks.

---

11. Command System

11.1 Command Format

```
/plugin.command --context=value --output=value
```

Examples:

```
/cos.weekly-review
/kk.daily-plan
/hv.analyze-deal --city=den-haag --source=funda
/ea.write-hld --client=bpost --topic=platform-engineering
/pai.create-prd --product=ros
/mkt.create-linkedin-post --topic=cloud-governance
/fin.prepare-compliance-pack --context=bank-onboarding
```

11.2 Command Anatomy

Every command file must define:

- Owner plugin
- Trigger
- Required inputs
- Optional inputs
- Skills used
- Connectors used
- Identity used
- Output asset
- Workbook used, if applicable
- Notion update rule
- Obsidian update rule
- Google Workspace action
- Safety level
- Confirmation rule
- Audit rule

11.3 Command Registry Example

Command	Plugin	Skills	Output	Workbook	Connectors
`/cos.weekly-review`	COS	priority-ranking, risk-review	Weekly Plan	Priority Matrix	Notion, Calendar
`/kk.daily-plan`	KK	calendar-manager, task-extractor	Daily Plan	None	Calendar, Notion
`/hv.analyze-deal`	HV	deal-analyzer, permit-pathway	Deal Memo	BRRRR Model	Funda, Notion
`/ea.write-adr`	EA	adr-writer	ADR	None	Drive, Notion
`/pai.create-prd`	PAI	prd-generator	PRD	Feature Matrix	Notion, GitHub
`/fin.track-insurance`	FIN	insurance-checker	Insurance Tracker	Insurance Comparison	Gmail, Drive

12. Skill System

A skill is a reusable capability. Skills must be small, clear, and testable.

Each skill file must define:

- Purpose
- Owner plugin
- Input requirements
- Process
- Output format
- Allowed connectors
- Safety level
- Related templates
- Related commands

12.1 Shared Skills

```
/03_SHARED/skills/
  email-composer.md
  meeting-summarizer.md
  task-extractor.md
  document-generator.md
  dashboard-updater.md
  notion-sync.md
  workbook-generator.md
```

```
decision-log-writer.md
research-brief-builder.md
risk-register-updater.md
```

12.2 Skill Boundary Rule

Skills do not own strategy.
Skills do not own memory.
Skills do not decide identity.
Skills are invoked by commands or plugins.

---

13. Workbook System

A workbook is a structured decision model.

13.1 Workbook Trigger Rule

Use a workbook when the task requires:

- Calculation
- Ranking
- Scenario comparison
- Financial modeling
- Cost modeling
- Prioritization
- Risk scoring
- Structured assumptions

13.2 Workbook Standard

Every workbook must include:

1. Assumptions
2. Inputs
3. Calculations
4. Scenarios
5. Sensitivity analysis
6. Recommendation
7. Decision threshold
8. Source references
9. Owner plugin
10. Related dashboard

13.3 Required Workbooks by Plugin

Plugin	Workbooks
HV	BRRRR Model, DSCR/LTV Model, Renovation Budget, Vendor Comparison, WWS Impact, Permit Scenario
EA	Cloud Cost Model, Migration Wave Plan, App Portfolio Assessment, Security Control Mapping, Vendor Comparison
PAI	Feature Prioritization, Pricing Model, Market Sizing, MVP Scope Matrix, Build vs Buy
MKT	Campaign ROI, Content Performance, Funnel Conversion, Launch Budget
FIN	Insurance Comparison, Invoice Tracker, Contract Obligation Tracker, Compliance Evidence Tracker
COS	Priority Matrix, Portfolio Review, Agent Health Scorecard
KK	Daily Plan, Follow-Up Tracker, Document Checklist

---

14. Template System

Templates are mandatory for repeatable outputs.

14.1 Core Templates

Plugin	Templates
COS	Strategy Memo, Weekly Review, Monthly Portfolio Review, Agent Spec, Skill Spec, Decision Memo
KK	Daily Plan, Meeting Brief, Follow-Up List, Document Checklist, Email Draft
HV	Deal Memo, Investment Scorecard, Lender Pack, Architect Brief, Contractor RFQ
EA	HLD, ADR, Architecture Review, Security Review, Client Meeting Brief, Interview Brief
PAI	PRD, MVP Plan, Agent Spec, Roadmap, Build Spec, Investor Memo
MKT	LinkedIn Post, Newsletter, Landing Page, Campaign Plan, Content Repurposing Plan
FIN	Insurance Summary, Contract Summary, Compliance Pack, Admin Email

14.2 Template Rule

```
If a repeatable output exists, never start from blank.
```

---

15. Obsidian Architecture

Obsidian is the local knowledge vault and thinking layer.

15.1 Obsidian Responsibilities

Use Obsidian for:

- Agent instructions
- Role memory
- Domain principles
- Research notes
- Client context
- Reusable templates
- Decision reasoning
- Strategic thinking
- Prompt libraries
- Meeting intelligence

15.2 Obsidian Vault Structure

```
/Obsidian Vault - ROS
│
├── 00_System
│   ├── ROS Principles.md
│   ├── Routing Model.md
│   ├── Identity Rules.md
│   ├── Action Safety.md
│   └── Prompt Standards.md
│
├── 01_Plugins
│   ├── COS
│   ├── KK
│   ├── HV
│   ├── EA
│   ├── PAI
│   ├── MKT
│   └── FIN
│
├── 02_Knowledge
│   ├── Enterprise Architecture
│   ├── Real Estate
│   ├── AI Products
│   ├── Marketing
│   └── Finance Admin
│
```

```
├── 03_Templates
│   ├── Meeting Note.md
│   ├── Decision Memo.md
│   ├── PRD.md
│   ├── HLD.md
│   ├── ADR.md
│   ├── Deal Memo.md
│   └── Weekly Review.md
│
├── 04_Inbox
│   ├── Voice Notes
│   ├── Raw Notes
│   ├── Web Clippings
│   └── Meeting Dumps
│
└── 05_Published Assets
    ├── To Notion
    ├── To Google Drive
    └── To Claude Artifacts
```

15.3 Obsidian Rule

```
Obsidian is the source of truth for knowledge and instructions.
Notion is the source of truth for operations and dashboards.
Google Drive is the source of truth for final shareable files.
```

---

16. Notion Architecture

Notion is the operational control plane.

16.1 Required Notion Databases

1. Plugin Registry
2. Skill Registry
3. Command Registry
4. Asset Library
5. Task Registry
6. Decision Log
7. Meeting Notes
8. Workbook Registry
9. Connector Registry
10. Audit Log

16.2 Master Dashboard

```
ROS Command Center
│
├── Today
├── This Week
├── Waiting For
├── Decisions Needed
├── Blocked Items
├── Active Assets
├── Active Workbooks
├── Open Follow-ups
├── Plugin Health
└── Audit Log
```

16.3 Domain Dashboards

Dashboard	Purpose
COS Command Center	Strategy, priorities, agent health, operating cadence
KK Personal Command Center	Today, inbox, follow-ups, documents, calendar
HV Deal Pipeline	Deals, financing, permits, renovation, vendors
EA Client Dashboard	Clients, HLDs, ADRs, risks, decisions, meetings
PAI Product Roadmap	Products, PRDs, features, experiments, backlog
MKT Content Calendar	Campaigns, posts, newsletters, landing pages
FIN Admin Dashboard	Insurance, invoices, contracts, documents, compliance

16.4 Asset Library Properties

Property	Type
Asset Name	Title
Asset Type	Select
Owner Plugin	Relation
Domain	Select
Status	Select
Version	Text
Link	URL

Property	Type
Source	Select
Related Command	Relation
Related Workbook	Relation
Last Updated	Date
Confidentiality	Select
Next Action	Text

---

17. Google Workspace Architecture

Google Workspace is the execution layer.

17.1 Google Workspace Capabilities

Product	ROS Use
Gmail	Inbox triage, summaries, drafts, follow-ups, sending after approval
Calendar	Daily planning, meeting prep, time blocking, review cadence
Drive	Store final assets, evidence, documents, workbooks
Docs	HLDs, PRDs, reports, client-ready documents
Sheets	Workbooks, trackers, financial models, prioritization matrices
Slides	Executive decks, strategy briefings, client presentations

17.2 Identity Registry

Identity	Primary Use	Allowed Plugins
`bguy.rubin@gmail.com`	Personal, EA, career, general operations	COS, KK, EA, MKT, FIN general
`bhollandvest@gmail.com`	HollandVest real estate	HV, FIN for HollandVest
`josephdoronrubin@gmail.com`	Restricted/unconfirmed	Disabled until confirmed
Future Workspace identities	Business-specific operations	To be mapped by identity policy

17.3 External Action Rule

```
Drafting is allowed.
Sending is gated.
Deleting is gated.
Submitting forms is gated.
Sharing documents is gated.
Financial/legal actions are gated.
```

---

18. Tool Diagnostic Architecture

ROS must include a tool diagnostic capability. This ensures the system knows whether the tools it wants to use are available, authorized, healthy, and appropriate before execution.

18.1 Tool Diagnostic Purpose

The diagnostic layer answers:

```
Can this plugin use this tool, with this identity, for this action, safely and
successfully?
```

18.2 Required Tool Diagnostic Files

```
/06_TOOL_DIAGNOSTICS/
  tool-registry.md
  connector-health.md
  identity-checks.md
  permission-checks.md
  failure-log.md
  fallback-routes.md
```

18.3 Tool Registry Fields

Field	Purpose
Tool Name	Gmail, Notion, Drive, Calendar, Obsidian, GitHub, Funda, etc.
Tool Type	Connector, API, MCP, browser, local script, scraper, workbook engine
Allowed Plugins	Which plugins can use it
Required Identity	Which account is required

Field	Purpose
Supported Actions	Read, summarize, draft, update, send, delete, submit
Safety Level	Highest permitted action level
Permission Status	Connected, missing, expired, restricted, unknown
Last Health Check	Last diagnostic timestamp
Fallback Tool	What to use if unavailable
Notes	Known limitations

18.4 Diagnostic Command Examples

```
/ros.diagnose-tools
/ros.check-connector --tool=gmail --identity=bguy.rubin@gmail.com
/ros.check-permissions --plugin=HV --tool=notion
/ros.test-command-readiness --command=/hv.analyze-deal
/ros.list-fallbacks --tool=google-drive
```

18.5 Tool Failure Protocol

When a tool fails, ROS must not hallucinate success.

It must report:

- What failed
- Which plugin was acting
- Which identity was used
- What action was attempted
- Whether data was changed
- Recommended fallback
- Whether user confirmation is needed

18.6 Fallback Examples

Failed Tool	Fallback
Gmail send unavailable	Create draft text asset only
Notion unavailable	Write local Markdown update queue
Google Drive unavailable	Save asset locally in `/04_ASSETS/`
Funda scraper unavailable	Use manual listing paste workflow
Calendar unavailable	Create proposed schedule note

Failed Tool	Fallback
Workbook engine unavailable	Generate CSV/Markdown model

---

19. Hybrid LLM Runtime Architecture

ROS must support multiple LLMs working on the same agent file system.

The system should be model-agnostic. The file system defines the role, memory, commands, templates, and policies. The LLM is the runtime executing the work.

19.1 Runtime Principle

```
The agent is the file-system package.
The model is the execution engine.
The connector is the system access layer.
The audit log records what happened.
```

19.2 Model Registry

Create:

```
/07_LLM_RUNTIME/model-registry.md
```

Required fields:

Field	Purpose
Model Name	Claude, Hermes, GPT, local model, specialist model
Provider	Anthropic, OpenAI, local, custom, other
Best Use	Strategy, writing, coding, daily assistant, classification, summarization
Allowed Plugins	Which plugins can use this model
Allowed Commands	Which commands it can run
Tool Access	Which tools it can call directly or indirectly
Data Sensitivity	Public, internal, confidential, client-specific, restricted
Cost Profile	Low, medium, high
Latency Profile	Fast, standard, slow

Field	Purpose
Fallback Model	Backup model

19.3 Recommended Runtime Routing

Use Case	Preferred Runtime
Strategy, architecture, PRDs	Claude Co-Work
Code implementation and repository changes	Claude Code
Personal assistant daily operations	Hermes or Claude Co-Work
Fast classification/routing	Lightweight local or low-cost model
Coding/build specs	Claude Code, GPT, or coding-specialist model
Private local summarization	Local LLM
Research synthesis	Claude or GPT
Workbook generation	Claude/GPT + spreadsheet engine
Tool diagnostics	Hermes, Claude Code, or lightweight diagnostic runner
GitHub issue/PR operations	Claude Code or GitHub-connected runtime

19.4 Hermes Integration Example

Hermes is a first-class ROS contributor. Hermes can act as the runtime for KK while using the same KK plugin file system.

```
Runtime: Hermes
Plugin: KK
Role: Personal Assistant
Allowed Commands:
- /kk.daily-plan
- /kk.inbox-triage
- /kk.follow-up-review
- /kk.prepare-meeting

Allowed Connectors:
- Calendar read
- Gmail read/summarize/draft
- Notion task updates, if authorized

Not Allowed:
```

```
- Send email without confirmation
- Delete files
- Submit forms
- Access EA client memory unless delegated
```

19.5 Multi-LLM Handoff Protocol

When one model hands work to another, the handoff must include:

- Source model
- Target model
- Plugin
- Command
- Current state
- Inputs
- Constraints
- Safety level
- Required output
- Memory files used
- Connectors used
- Audit entry

19.6 Runtime Safety Rules

```
No model can bypass plugin boundaries.
No model can bypass identity policy.
No model can bypass client confidentiality.
No model can write memory without following memory policy.
No model can perform Level 4 or Level 5 actions without confirmation.
```

19.7 Multi-Model Evaluation

ROS should track model performance by command.

Create:

```
/07_LLM_RUNTIME/evaluation-log.md
```

Track:

- Command executed
- Model used
- Output quality
- Accuracy
- Speed

- Cost
- Tool success/failure
- User correction required
- Recommended future model

---

20. GitHub Architecture

GitHub is the source-control, collaboration, and release backbone for ROS.

ROS must use GitHub to manage the file-system agent architecture as a real software product, not as a pile of Markdown files.

20.1 GitHub Responsibilities

Use GitHub for:

- Version control of all ROS files
- Branches for new plugins, commands, skills, and templates
- Pull requests for reviewable agent behavior changes
- Issues for backlog, defects, and improvements
- Releases for stable ROS versions
- GitHub Actions for validation checks
- CODEOWNERS for responsibility boundaries
- Environment configuration examples
- Change audit through commit history

20.2 Repository Structure

```
ros/
│
├── .github/
│   ├── workflows/
│   │   ├── validate-ros.yml
│   │   ├── lint-markdown.yml
│   │   ├── check-command-registry.yml
│   │   ├── check-plugin-structure.yml
│   │   └── environment-diagnostics.yml
│   │
│   ├── ISSUE_TEMPLATE/
│   │   ├── command-request.md
│   │   ├── plugin-change.md
│   │   ├── tool-failure.md
│   │   └── bug-report.md
│   │
│   └── PULL_REQUEST_TEMPLATE.md
```

```
│
├── 00_SYSTEM/
├── 01_ROOT_PLUGINS/
├── 02_DOMAIN_PLUGINS/
├── 03_SHARED/
├── 04_ASSETS/
├── 05_AUDIT/
├── 06_TOOL_DIAGNOSTICS/
├── 07_LLM_RUNTIME/
├── 08_GITHUB/
└── 09_ENVIRONMENTS/
```

20.3 Branching Strategy

```
main = stable production ROS
stage = validated pre-production ROS
dev = active integration work
feature/plugin-{name} = new or changed plugin
feature/command-{name} = new or changed command
fix/tool-{name} = connector or tool fix
experiment/{topic} = temporary research or prototype
```

20.4 Pull Request Rules

Every PR must state:

- What changed
- Which plugin is affected
- Which command/skill/template is affected
- Whether memory behavior changed
- Whether identity/tool access changed
- Safety impact
- Test/dry-run result
- Rollback plan

20.5 GitHub Issue Types

```
Plugin Request
Command Request
Skill Request
Template Request
Workbook Request
Tool Failure
Environment Issue
Identity Issue
```

```
Bug
Improvement
```

20.6 GitHub Actions Validation

GitHub Actions should validate:

- Required plugin folders exist
- CLAUDE.md exists for every plugin
- MEMORY.md exists for every plugin
- Commands follow `/plugin.command` naming
- Command files define owner, inputs, outputs, safety level, and audit rule
- No duplicate command names exist
- Markdown files follow smart/efficient instruction principles
- Environment files do not contain secrets
- Connector references exist in Tool Registry
- Model references exist in Model Registry

20.7 CODEOWNERS Principle

Use CODEOWNERS to define responsibility boundaries:

```
/01_ROOT_PLUGINS/COS/        @guy
/01_ROOT_PLUGINS/KK/         @guy
/02_DOMAIN_PLUGINS/HV/       @guy
/02_DOMAIN_PLUGINS/EA/       @guy
/02_DOMAIN_PLUGINS/PAI/      @guy
/06_TOOL_DIAGNOSTICS/        @guy
/07_LLM_RUNTIME/             @guy
/09_ENVIRONMENTS/            @guy
```

Future contributors, including Hermes-driven or Claude Code-driven automation, must work through branches and PRs unless explicitly authorized.

---

21. Environment Management Architecture

ROS must support safe execution across environments.

The goal is simple: test agent behavior before it touches real Gmail, Notion, Drive, Calendar, GitHub, finance documents, or client data.

21.1 Required Environments

Environment	Purpose	Tool Access
local	File editing, Obsidian work, local tests	No real external actions
sandbox	Simulated tool execution and dry runs	Mock connectors or test accounts
dev	Connected test integrations	Limited test data only
stage	Pre-production validation	Real structure, restricted actions
prod	Operational use	Real identities and data with safety gates

21.2 Environment Registry

Create:

```
/09_ENVIRONMENTS/environment-registry.md
```

Required fields:

Field	Purpose
Environment Name	local, sandbox, dev, stage, prod
Purpose	Why it exists
Allowed Plugins	Which plugins can run here
Allowed Commands	Which commands can run here
Allowed Tools	Which tools/connectors are enabled
Allowed Identities	Which accounts may be used
Data Type	mock, test, internal, client, production
External Actions	blocked, draft-only, restricted, allowed-with-confirmation
Promotion Criteria	What must pass before moving up

21.3 Environment Files

Use example files only. Never commit real secrets.

```
/09_ENVIRONMENTS/
  local.env.example
  sandbox.env.example
  dev.env.example
```

```
  stage.env.example
  prod.env.example
```

Example variables:

```
ROS_ENV=sandbox
ROS_DEFAULT_RUNTIME=hermes
ROS_ORCHESTRATOR=claude-cowork
ROS_CODE_RUNTIME=claude-code
ROS_NOTION_MODE=dry-run
ROS_GMAIL_MODE=draft-only
ROS_GOOGLE_DRIVE_MODE=read-write-test
ROS_GITHUB_MODE=branch-only
ROS_EXTERNAL_ACTIONS=blocked
```

21.4 Secrets Policy

```
Never store secrets in Markdown.
Never store secrets in GitHub commits.
Never store production tokens in Obsidian or Notion.
Use environment variables, secret managers, or GitHub Actions secrets.
```

21.5 Promotion Policy

A command can move from sandbox to production only when:

1. Command file is complete.
2. Tool diagnostic passes.
3. Identity mapping is correct.
4. Safety level is defined.
5. Dry-run output is reviewed.
6. Audit rule exists.
7. Rollback/fallback exists.
8. PR is reviewed and merged.

21.6 Environment Diagnostic Commands

```
/ros.env-status
/ros.env-check --env=sandbox
/ros.env-promote --command=/kk.daily-plan --from=sandbox --to=dev
/ros.env-dry-run --command=/hv.analyze-deal --env=sandbox
/ros.env-validate-secrets
```

---

22. Sync Architecture

ROS must support controlled sync between the local file system, GitHub, Obsidian, Notion, Google Workspace, Claude Code, Claude Co-Work, and Hermes.

The goal is not maximum sync. The goal is reliable sync.

22.1 Source of Truth Map

System	Source of Truth For	Notes
GitHub	ROS file-system architecture, plugins, commands, skills, templates, policies	Canonical versioned source for the system
Obsidian	Durable knowledge, thinking, principles, research notes, prompt libraries	Local-first knowledge vault
Notion	Operational state, dashboards, registries, tasks, decisions, asset index	Cockpit, not deep knowledge base
Google Drive	Final shareable assets and evidence documents	Docs, Sheets, Slides, PDFs, client-ready files
Google Calendar	Time commitments and review cadence	Execution schedule
Gmail	Communication history and outbound drafts/sends	Identity-controlled execution
Hermes	Daily assistant runtime, cron jobs, lightweight automation, messaging interface	Main runtime for KK workflows
Claude Code	Repository changes, validation, automation scripts, GitHub operations	Implementation contributor
Claude Co-Work	Strategy, planning, reasoning, architecture, orchestration	Executive orchestration contributor

22.2 Sync Lanes

ROS uses sync lanes instead of uncontrolled two-way sync.

```
Lane 1 — GitHub ↔ Local File System
Purpose: version-controlled edits to ROS files.
Primary runtime: Claude Code, Hermes, or manual Git.

Lane 2 — Obsidian → GitHub
Purpose: publish approved Markdown knowledge/instructions into ROS repository.
Mode: controlled commit/PR.
```

```
Lane 3 — GitHub → Obsidian
Purpose: pull approved system files into the local knowledge vault.
Mode: read/sync only, no silent overwrite.

Lane 4 — GitHub → Notion
Purpose: update registries and dashboards from approved plugin/command/asset
metadata.
Mode: automated or semi-automated after validation.

Lane 5 — Notion → GitHub
Purpose: convert approved tasks, command requests, or issues into GitHub issues.
Mode: explicit command only.

Lane 6 — Google Workspace → Asset Library
Purpose: index final Docs, Sheets, Slides, PDFs, and email drafts in Notion.
Mode: metadata sync, not full content duplication.

Lane 7 — Hermes → GitHub/Notion
Purpose: scheduled backups, daily assistant updates, cron-generated logs,
operational notes.
Mode: branch/PR for system changes; direct Notion update only for allowed
operational records.
```

22.3 Required Sync Files

Create:

```
/10_SYNC/
  sync-policy.md
  source-of-truth-map.md
  github-sync.md
  obsidian-sync.md
  notion-sync.md
  google-workspace-sync.md
  hermes-cron-jobs.md
  conflict-resolution.md
  sync-audit-log.md
```

22.4 Hermes Cron Jobs

Hermes should run lightweight scheduled operations, especially for KK.

Initial cron jobs:

```
hermes.daily-briefing
hermes.github-backup
hermes.notion-dashboard-refresh
hermes.follow-up-scan
hermes.tool-health-check
hermes.sync-audit-summary
```

Cron rules:

- Cron jobs may read approved sources.
- Cron jobs may update operational Notion records only if authorized.
- Cron jobs may create GitHub branches or issues.
- Cron jobs must not modify production plugin behavior directly.
- Cron jobs must not send email, submit forms, delete files, or share documents without confirmation.

22.5 Claude Code Role in Sync

Claude Code is the implementation contributor.

Use Claude Code for:

- editing ROS repository files
- creating plugin folders
- creating command/skill/template files
- writing validation scripts
- updating GitHub Actions
- resolving markdown lint issues
- opening PRs
- running local diagnostics

Claude Code must not bypass GitHub review for production-impacting changes.

22.6 Sync Conflict Policy

When two systems disagree, use this order:

```
1. GitHub for system files and plugin definitions
2. Obsidian for durable knowledge and thinking notes
3. Notion for operational state
4. Google Workspace for final files
5. Audit logs for what happened
```

If conflict affects identity, client data, safety level, tool permissions, or production behavior, stop and request approval.

22.7 Sync Commands

```
/ros.sync-status
/ros.sync-github
/ros.sync-obsidian-to-github
/ros.sync-github-to-notion
/ros.sync-google-assets
/ros.create-github-issue-from-notion
/ros.run-hermes-cron-check
/ros.resolve-sync-conflict
```

---

23. Safety and Confirmation Model

Level	Action Type	Confirmation Required
Level 0	Think, classify, analyze	No
Level 1	Read and summarize	No
Level 2	Draft asset or internal output	Usually no
Level 3	Update Notion, Drive, Calendar	Prefer confirmation unless pre-authorized
Level 4	Send email, forward files, publish	Yes
Level 5	Submit form, legal, financial, delete/share sensitive data	Explicit confirmation required

18.1 Mandatory Confirmation Prompt

Before Level 4 or Level 5 action, ROS must show:

- Acting plugin
- Identity/account used
- Recipient/system
- Exact action
- Content or payload summary
- Risk level
- Confirmation request

---

24. Memory Model

19.1 Memory Types

Memory Type	Purpose	Location
Global Memory	Stable user preferences and system-wide context	`/ROS/MEMORY.md`
Plugin Memory	Domain-specific durable facts	`/plugins/{plugin}/MEMORY.md`
Client Memory	Client-specific EA context	`/plugins/EA/clients/{client}/MEMORY.md`
Project Memory	Product, deal, or campaign context	Project folder
Session Memory	Temporary context during active work	Session log

19.2 Memory Rules

- MEMORY.md must contain only durable context.
- Do not store random conversation history.
- Do not duplicate global rules inside plugin memory.
- EA client memory must remain isolated by client.
- Temporary notes go to session logs or inbox, not memory.

---

25. Smart & Efficient Markdown Principle

Create this file:

```
/00_SYSTEM/markdown-instruction-principles.md
```

20.1 Core Rule

All ROS Markdown files, CLAUDE.md files, MEMORY.md files, command files, skill files, templates, and operating instructions must be smart, efficient, modular, and execution-oriented.

```
Do not write long instruction files unless complexity truly requires it.
Every instruction must earn its place.
```

20.2 Markdown Principles

1. Be concise.
2. Be modular.
3. Be executable.
4. Avoid repetition.
5. Prefer decision rules over explanations.
6. Prefer checklists over essays.
7. Use command-first design.
8. Use templates for repeatable outputs.
9. Use workbooks for decisions involving money, risk, scoring, ranking, or scenarios.
10. Keep memory clean.
11. Separate principles from procedures.
12. Optimize for retrieval.
13. No bloated agent files.
14. No vague instructions.
15. Every file must answer:
    - What is this?
    - Who owns it?
    - When is it used?
    - What inputs are needed?
    - What output is expected?
    - What safety rule applies?

20.3 Instruction Quality Acceptance Criteria

A ROS Markdown file is acceptable only if:

1. It has one clear purpose.
2. It is easy to scan.
3. It avoids duplicate rules.
4. It defines clear inputs and outputs.
5. It supports execution, not theory.
6. It is short enough for the agent to use reliably.
7. It links to shared policies instead of repeating them.

---

26. Operating Cadence

21.1 Daily

Owned by KK.

```
/kk.daily-plan
/kk.inbox-triage
/kk.follow-up-review
```

Outputs:

- Daily plan
- Top priorities
- Meetings to prepare
- Follow-ups
- Missing documents
- Waiting-for list

21.2 Weekly

Owned by COS.

```
/cos.weekly-review
```

Outputs:

- Weekly priority stack
- Domain review
- Blockers
- Decisions needed
- Kill/defer list
- Next automation candidate

21.3 Monthly

Owned by COS.

```
/cos.monthly-portfolio-review
```

Outputs:

- Portfolio review
- Domain performance
- Financial/strategic impact
- Agent health
- System improvement backlog

---

27. Example End-to-End Flows

22.1 HollandVest Deal Flow

User request:

```
Analyze this Funda property and ask Ahmad and Ilia for feedback.
```

Routing:

```
Domain: HollandVest
Plugin: HV
Command: /hv.analyze-deal
Skills:
- deal-analyzer
- permit-pathway
- financing-pack-builder
- email-composer

Connectors:
- Funda
- Notion HV dashboard
- Gmail bhollandvest@gmail.com

Workbooks:
- BRRRR Model
- DSCR/LTV Model

Outputs:
- Deal scorecard
- Investment memo
- Financing questions
- Architect questions
- Draft emails

Safety:
- Analysis = allowed
- Draft email = allowed
- Send email = Level 4, confirmation required
```

22.2 EA Client HLD Flow

User request:

```
Prepare an HLD for bpost platform engineering.
```

Routing:

```
Domain: Enterprise Architecture
Plugin: EA
Client: bpost
Command: /ea.write-hld --client=bpost

Skills:
- hld-writer
- platform-engineering-advisor
- cloud-governance-review

Connectors:
- Notion EA
- Google Drive EA

Output:
- HLD artifact
- Architecture decision update
- Risk register update

Safety:
- Internal document creation allowed
- Sharing externally requires confirmation
```

22.3 Weekly Planning Flow

User request:

```
What should I focus on this week?
```

Routing:

```
Domain: Cross-domain
Plugin: COS
Command: /cos.weekly-review

Skills:
- priority-ranking
- risk-review
- decision-review
```

```
Connectors:
- Notion Command Center
- Calendar
- Task Registry

Output:
- Weekly focus plan
- Top five priorities
- Kill/defer list
- Decisions needed
```

---

28. MVP Scope

Phase 1 — Core ROS Foundation

Deliverables:

```
/ROS/CLAUDE.md
/ROS/MEMORY.md
/00_SYSTEM/principles.md
/00_SYSTEM/routing.md
/00_SYSTEM/plugin-policy.md
/00_SYSTEM/command-policy.md
/00_SYSTEM/workbook-policy.md
/00_SYSTEM/identity-policy.md
/00_SYSTEM/markdown-instruction-principles.md
/06_TOOL_DIAGNOSTICS/tool-registry.md
/06_TOOL_DIAGNOSTICS/connector-health.md
/07_LLM_RUNTIME/model-registry.md
/07_LLM_RUNTIME/routing-rules.md
/08_GITHUB/repository-policy.md
/08_GITHUB/branching-strategy.md
/09_ENVIRONMENTS/environment-registry.md
/09_ENVIRONMENTS/secrets-policy.md
/10_SYNC/sync-policy.md
/10_SYNC/source-of-truth-map.md
/10_SYNC/hermes-cron-jobs.md
```

Phase 2 — Root Plugins

Deliverables:

```
/01_ROOT_PLUGINS/COS/CLAUDE.md
/01_ROOT_PLUGINS/COS/commands/weekly-review.md
/01_ROOT_PLUGINS/COS/commands/design-agent.md

/01_ROOT_PLUGINS/KK/CLAUDE.md
/01_ROOT_PLUGINS/KK/commands/daily-plan.md
/01_ROOT_PLUGINS/KK/commands/inbox-triage.md
```

Phase 3 — High-Value Domain Plugins

Deliverables:

```
/02_DOMAIN_PLUGINS/HV/CLAUDE.md
/02_DOMAIN_PLUGINS/HV/commands/analyze-deal.md
/02_DOMAIN_PLUGINS/HV/workbooks/brrrr-model.md

/02_DOMAIN_PLUGINS/EA/CLAUDE.md
/02_DOMAIN_PLUGINS/EA/commands/write-hld.md
/02_DOMAIN_PLUGINS/EA/commands/prepare-client-meeting.md
```

Phase 4 — Notion Control Plane

Deliverables:

```
Plugin Registry
Skill Registry
Command Registry
Asset Library
Task Registry
Decision Log
Meeting Notes
Workbook Registry
Connector Registry
Audit Log
Command Center Dashboard
```

Phase 5 — Growth Plugins

Deliverables:

```
/02_DOMAIN_PLUGINS/PAI/CLAUDE.md
/02_DOMAIN_PLUGINS/MKT/CLAUDE.md
/02_DOMAIN_PLUGINS/FIN/CLAUDE.md
```

---

29. Acceptance Criteria

ROS is successful when:

1. COS and KK exist as root plugins.
2. HV, EA, PAI, MKT, and FIN exist as separate domain plugins.
3. Each plugin has its own CLAUDE.md and MEMORY.md.
4. Each plugin has its own skills, commands, templates, workbooks, connectors, dashboards, assets, and audit folder.
5. Commands follow `/plugin.command` syntax.
6. Workbooks are first-class decision models.
7. EA supports multiple clients with strict memory separation.
8. Gmail and Google Workspace identities are mapped to plugins.
9. Notion acts as the operational control plane.
10. Obsidian acts as the durable knowledge and instruction layer.
11. Google Workspace acts as the execution and collaboration layer.
12. External actions require explicit confirmation.
13. Markdown instructions are concise, modular, and execution-oriented.
14. The system never collapses into one plugin.
15. Every asset has an owner, status, source, and next action.
16. ROS can diagnose whether a tool, connector, identity, or command is ready before execution.
17. ROS can support multiple LLM runtimes working on the same file-system agent architecture.
18. Hermes can be assigned as the runtime for KK without changing KK's plugin structure.
19. Multi-model handoffs are logged and constrained by plugin, identity, memory, and safety policies.
20. GitHub stores and versions the ROS file-system agent architecture.
21. New plugins, commands, skills, and templates are changed through branches and pull requests.
22. GitHub Actions validate plugin structure, command completeness, Markdown quality, and environment safety.
23. ROS supports local, sandbox, dev, stage, and prod environments.
24. No production connector or command is promoted without diagnostic checks and review.
25. GitHub, Obsidian, Notion, Google Workspace, Claude Code, Claude Co-Work, and Hermes are connected through explicit sync lanes.
26. Hermes can run scheduled jobs for backups, daily briefings, tool checks, and dashboard refreshes.
27. Claude Code can implement repository changes through branches and PRs.
28. Sync conflicts have a defined source-of-truth hierarchy and escalation rule.

---

30. Build Instruction for Claude Co-Work

Use this as the primary implementation instruction:

```
Build ROS as a file-system-based AI operating system for Claude Co-Work.

Do not build ROS as one plugin.

Each major role must be implemented as a separate plugin:
- COS = Chief of Staff
- KK = Personal Assistant
- HV = HollandVest Real Estate Development
- EA = Enterprise Architecture Consulting
- PAI = AI Product Team
- MKT = Marketing / Content
- FIN = Finance & Admin

COS and KK are root plugins.
HV, EA, PAI, MKT, and FIN are domain plugins.

Each plugin must contain:
- CLAUDE.md
- MEMORY.md
- skills/
- commands/
- templates/
- workbooks/
- connectors/
- dashboards/
- assets/
- audit/

Commands must use:
/plugin.command

Workbooks are first-class decision models.

Obsidian is the durable knowledge and instruction layer.
Notion is the operational dashboard and database layer.
Google Workspace is the execution and collaboration layer.
Claude Co-Work is the reasoning and orchestration layer.

The system must enforce:
- role separation
- plugin separation
- identity routing
- client data isolation
- action safety levels
- confirmation gates
- template-first outputs
- workbook-driven decisions
```

```
- tool diagnostics
- hybrid LLM runtime routing
- multi-model handoff protocol
- GitHub-backed source control and PR workflow
- environment-managed execution across local, sandbox, dev, stage, and prod
- GitHub/Obsidian/Notion/Google Workspace sync lanes
- Hermes cron jobs and operational automation
- Claude Code repository implementation workflow
- sync conflict resolution
- audit logging
- smart and efficient Markdown instructions

All Markdown files must be concise, modular, and execution-oriented.
Every instruction must earn its place.
```

---

31. Immediate Next Steps

Step 1 — Generate Core System Files

Create:

```
/ROS/CLAUDE.md
/ROS/MEMORY.md
/00_SYSTEM/principles.md
/00_SYSTEM/routing.md
/00_SYSTEM/plugin-policy.md
/00_SYSTEM/command-policy.md
/00_SYSTEM/workbook-policy.md
/00_SYSTEM/identity-policy.md
/00_SYSTEM/markdown-instruction-principles.md
/06_TOOL_DIAGNOSTICS/tool-registry.md
/06_TOOL_DIAGNOSTICS/connector-health.md
/06_TOOL_DIAGNOSTICS/fallback-routes.md
/07_LLM_RUNTIME/model-registry.md
/07_LLM_RUNTIME/routing-rules.md
/07_LLM_RUNTIME/handoff-protocol.md
/08_GITHUB/repository-policy.md
/08_GITHUB/branching-strategy.md
/08_GITHUB/github-actions.md
/09_ENVIRONMENTS/environment-registry.md
/09_ENVIRONMENTS/secrets-policy.md
/09_ENVIRONMENTS/promotion-policy.md
/10_SYNC/sync-policy.md
/10_SYNC/source-of-truth-map.md
```

```
/10_SYNC/hermes-cron-jobs.md
/10_SYNC/conflict-resolution.md
```

Step 2 — Generate Root Plugins

Create:

```
/01_ROOT_PLUGINS/COS/CLAUDE.md
/01_ROOT_PLUGINS/KK/CLAUDE.md
```

Step 3 — Generate HV and EA Plugins

Create:

```
/02_DOMAIN_PLUGINS/HV/CLAUDE.md
/02_DOMAIN_PLUGINS/EA/CLAUDE.md
```

Step 4 — Build Notion Control Plane

Create the ten required databases and the ROS Command Center dashboard.

Step 5 — Implement Commands

Start with:

```
/cos.weekly-review
/kk.daily-plan
/hv.analyze-deal
/ea.write-hld
/pai.create-prd
/fin.prepare-compliance-pack
```

---

32. Final Architecture Decision

ROS will use **Option A — Controlled One-Way Sync** for MVP:

```
Obsidian → Notion for selected operational assets only.
Notion does not write back automatically into Obsidian.
Google Drive stores final shareable files.
```

Rationale:

- Cleaner state management.
- Lower duplication risk.
- Better control over what becomes operational.
- Easier to audit.
- Better fit for MVP.

Two-way sync can be considered later after the plugin and registry model is stable.

Second final decision: ROS will use **explicit sync lanes** instead of uncontrolled global sync.

```
GitHub = canonical system version
Obsidian = durable thinking and knowledge
Notion = operational state
Google Workspace = final collaboration assets
Hermes = scheduled operational runtime
Claude Code = implementation and repository change runtime
Claude Co-Work = strategy and orchestration runtime
```

This keeps the system powerful without turning it into a synchronization mess.